# Lab 04 Conditional Access and workload identity

AI for IT Security Professionals | TGS-2023039344 | v6.0

## Purpose and output

Create a policy-mode comparison and workload identity diagnosis. This lab supports LO3. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

## Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use py -3 if python3 is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

## Folder contents

- mock-data.json: synthetic input with no real personal data or credentials.
- analyse.py: runnable analysis for this lab only.
- expected-results.json: expected baseline and source checksum.
- ai-review-prompt.txt: evidence-constrained AI review prompt.
- evidence-template.md: your deliverable template.
- INSTRUCTIONS.md and INSTRUCTIONS.pdf: the same procedure in two formats.

## 1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open analyse.py in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

## 2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open results.json. The expected decisions in output order are: WOULD_BLOCK, BLOCK, WRONG_AUDIENCE. The source_sha256 binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-
results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

## 3 Trace the mechanism

### *Conditional Access evaluation*

Report-only evaluates a policy without enforcing its access decision.
Contract: state=enabledForReportingButNotEnforced
Accepted case: An MFA policy records a would-block result for the test user.
Counterexample: A report-only result is presented as an enforced block.
Diagnosis: Pilot enforcement with a scoped group after reviewing emergency-access exclusions.
Evidence: Preserve policy mode and sign-in evaluation result, not only its name.

### *Managed identity token flow*

An Azure workload requests a token and the resource checks its audience and assigned permissions.
Contract: principal_id, token_audience, resource_scope
Accepted case: A workload identity has the required role on its target resource.
Counterexample: A valid token targets the wrong resource audience.
Diagnosis: Diagnose token audience separately from resource permission; never paste access tokens into AI tools.
Evidence: Record identity object ID, target resource and redacted error code.

### *Application consent boundaries*

API permission consent and Azure resource RBAC are separate grants.
Contract: app_id, service_principal_id, permission_type, consent
Accepted case: An app has only the delegated scope needed for the signed-in user.
Counterexample: Tenant-wide application permission is mistaken for a user-only grant.
Diagnosis: Review whether app-only access is required and remove unused consent through approval.
Evidence: Capture permission type, consenting authority and intended data boundary.

## 4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Set signin-1 mfa to true. The result changes to ALLOW in this simplified model; explain why this does not prove a real Conditional Access deployment.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

## 5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.
For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

## 6 Verify the corresponding operational control

### *Azure procedure B Report-only policy and application identity*

1. Confirm the training tenant and the approved test group. Verify emergency-access accounts are outside the test scope. Do not use All users or all production resources for this exercise.
2. Microsoft Entra admin center > Protection > Conditional Access > Policies > New policy. Name it ITSEC MFA test <initials>.
3. Under Users select only the approved training group. Under Target resources select Microsoft Azure Management where available. Under Grant select Require multifactor authentication. Set Enable policy to Report-only and create it. Capture the scope, grant and mode.
4. Use a trainer-approved test sign-in. Open Monitoring & health > Sign-in logs, select the event and inspect its Report-only evaluation. Record the policy result, timestamp and test-user identifier. Do not claim that a report-only result blocked access.
5. App registrations > New registration: create a single-tenant training app itsec-app-<initials> with only trainer-required configuration. Record application/client ID and directory ID. Do not create a client secret for this exercise.

6. Open API permissions. Distinguish delegated from application permissions and record existing consent state. Do not grant tenant-wide administrator consent for unapproved scopes. Record the exact consent blocker if relevant.
7. On the assigned Azure VM or supported application resource, open Identity > System assigned. If the trainer permits enabling it, save the setting and record the principal ID; otherwise inspect the preconfigured identity.
8. On the trainer-designated target resource, inspect IAM and the identity's assigned role and scope. Use the approved workload demonstration to capture a successful resource operation or redacted access-denied result. Record the target/audience separately from resource permission; never export the token.
9. Complete the comparison with the local fixture. Remove the lab-created report-only policy and app registration after approval, and restore identity configuration only if you changed it and the trainer authorises reversal.
For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

## 7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

## Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a policy-mode comparison and workload identity diagnosis with an owner, evidence and remaining uncertainty.

## Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with python3 -m json.tool changed-data.json; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

## References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.